

ALBERI DI RICERCA

Giovedì 12 Marzo 2026



DIZIONARI

- Gli alberi binari di ricerca sono usati per realizzare in modo efficiente il tipo di dato dizionario

tipo Dizionario:

dati: un insieme S di coppie $(elem, chiave)$

operazioni:

insert $(elem\ e, chiave\ k)$

aggiunge a S una nuova coppia (e, k)

delete $(elem\ e)$

cancella da S l'elemento e

search $(chiave\ k) \rightarrow elem$

se la chiave k è presente in S restituisce un elemento e ad essa associato,
e `null` altrimenti



QUALI ALTRE STRUTTURE DATI PER I DIZIONARI?

- Array ordinato
 - Ricerca $O(\log n)$, inserimento/cancellazione $O(n)$
- Lista non ordinata
 - Ricerca/cancellazione $O(n)$, inserimento $O(1)$

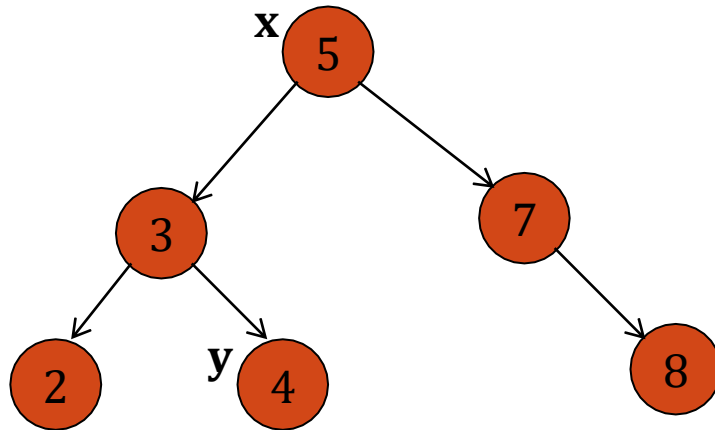


ALBERO BINARIO DI RICERCA (ABR)

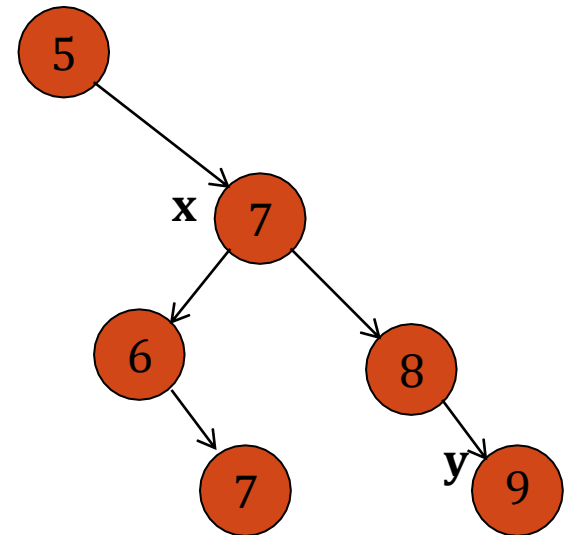
- Un albero binario nel quale sono memorizzati elementi (chiavi o key) di un insieme totalmente ordinato è un ABR se:
 - Per ogni nodo x dell'albero
 - Tutti i nodi appartenenti al **sottoalbero sinistro di x** hanno una chiave minore o uguale alla chiave di x , cioè $\text{key}[y] \leq \text{key}[x]$
 - Tutti i nodi appartenenti al **sottoalbero destro di x** hanno una chiave maggiore della chiave di x , cioè $\text{key}[y] > \text{key}[x]$



ESEMPI DI ABR



$\text{key}[y] \leq \text{key}[x]$

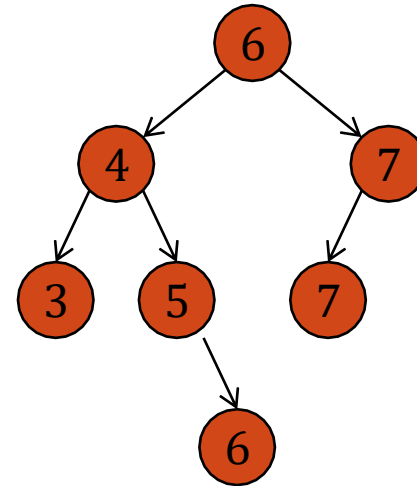
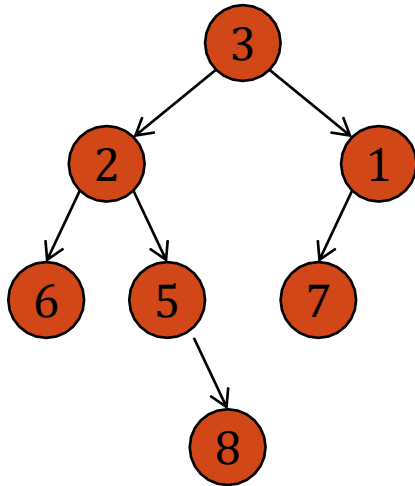


$\text{key}[x] > \text{key}[y]$

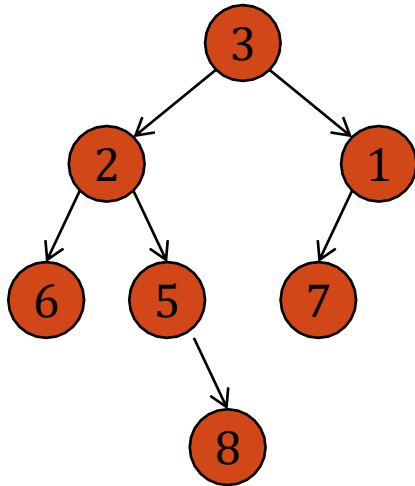


ESEMPIO DI ABR

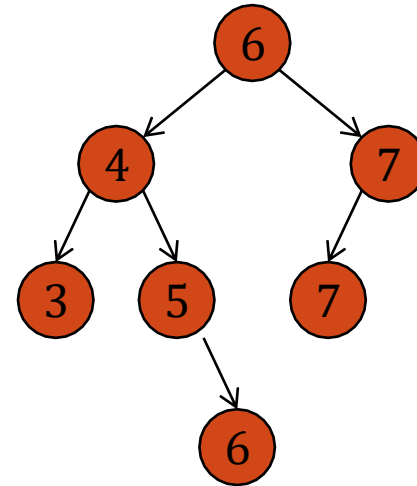
Quale tra i seguenti è un ABR?



ESEMPIO DI ABR



Non è un ABR



E' un ABR



Come implementare?

- Per ogni nodo è utile mantenere:
 - Chiave
 - Altri dati
 - Figlio sinistro
 - Figlio destro



Classe BSTnode

```
class BSTnode<K> {  
    private K key;  
    private BSTnode<K> left, right;  
  
    public BSTnode(K key, BSTnode<K> left, BSTnode<K> right) {  
        this.key = key;  
        this.left = left;  
        this.right = right;  
    }  
}
```



Classe BST

K deve implementare
un metodo
int compareTo(K altro)

```
public class BST<K extends Comparable<K>> {  
  
    private BSTnode<K> root; // ptr to the root of the BST  
  
    public BST() { root = null; }  
  
    public void insert(K key) { ... }  
  
    //aggiunge un nodo all'albero  
  
    public void delete(K key) { ... }  
  
    // cancella il nodo se è presente, altrimenti non fa nulla
```



Classe BST

```
public boolean lookup(K key) { ... }
```

```
// se la chiave è contenuta nell'ABR restituisce TRUE altrimenti FALSE
```

```
public void print(PrintStream p) { ... }
```

```
// Stampa I valori dell'ABR in ordine)
```

```
}
```



Per inserire anche altri dati (ciascuno con la sua chiave)

```
class BSTnode<K, V> {  
  
    private K key;  
  
    private V value;  
  
    private BSTnode<K, V> left, right;  
  
    // *** constructor ***  
  
    public BSTnode(K key, V value, BSTnode<K, V> left, BSTnode<K, V> right)  
    {  
        this.key = key;  
        this.value = value;  
    }  
}
```



Per inserire anche altri dati (ciascuno con la sua chiave)

```
this.left = left;
```

```
this.right = right;
```

```
}
```

```
}
```



Continua...

```
public class BST<K extends Comparable<K>, V> {  
  
    private BSTnode<K, V> root; // puntatore alla radice dell'albero  
  
    public BST() { root = null; }  
  
    public void insert(K key, V value) {...}  
  
    // aggiungi chiave e valore associato al BST;
```



Continua...

```
public void delete(K key) {...}
```

```
public V lookup(K key) {...}
```

```
    // se la chiave c'è ritorna il suo valore associate, altrimenti  
    restituisci NULL
```

```
public void print(PrintStream p) {...}
```

```
}
```



Ricerca di un elemento in un ABR

```
public boolean lookup(K key) {  
    return lookup(root, key);  
}
```



Esempio di implementazione di un ABR

```
- public class BST<Key extends Comparable<Key>, Value> {-           this.key = key;
-     private Node root;      // root of BST
-                               -           this.val = val;
-                               -           this.size = size;
-     private class Node {-           }
-     private Key key;      // sorted by key
-                               -           }
-     private Value val;    // associated data
-     private Node left, right; // left and right subtrees
-                               - public BST() {
-     private int size;      // number of nodes in subtree
-                               -           }

-     public Node(Key key, Value val, int size) {
```

CONTINUA....



Metodi

- `public boolean isEmpty() {`
- `return size() == 0;`
- `}`

- `public int size() {`
- `return size(root);`
- `}`

- `private int size(Node x) {`
- `if (x == null) return 0;`
- `else return x.size;`
- `}`



Ricerca nell'ABR

```
- public boolean contains(Key key) {  
-     if (key == null) throw new  
IllegalArgumentException("argument to  
contains() is null");  
-     return get(key) != null;  
- }  
  
- public Value get(Key key) {  
-     return get(root, key);  
- }
```

```
- private Value get(Node x, Key key) {  
-     if (key == null) throw new  
IllegalArgumentException("calls get() with a  
null key");  
-     if (x == null) return null;  
-     int cmp = key.compareTo(x.key);  
-     if (cmp < 0) return get(x.left, key);  
-     else if (cmp > 0) return get(x.right, key);  
-     else return x.val;  
- }
```

Complessità di tempo: $O(h)$ nel caso peggiore
 h altezza dell'ABR



Inserimento in un ABR

```
- public void put(Key key, Value val) {  
-     if (key == null) throw new  
IllegalArgumentException("calls put() with a null  
key");  
-     if (val == null) {  
-         delete(key);  
-         return;  
-     }  
-     root = put(root, key, val);  
- }
```

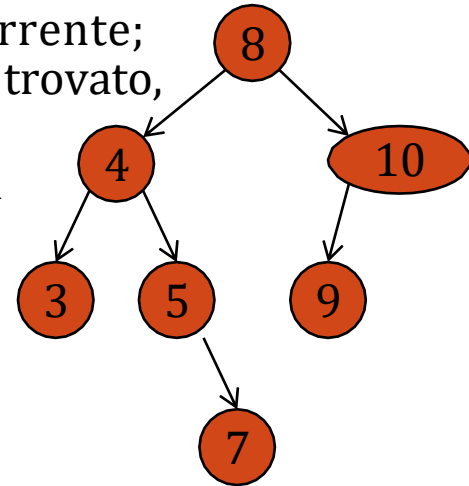
```
- private Node put(Node x, Key key, Value val) {  
-     if (x == null) return new Node(key, val, 1);  
-     int cmp = key.compareTo(x.key);  
-     if (cmp < 0) x.left = put(x.left, key, val);  
-     else if (cmp > 0) x.right = put(x.right, key, val);  
-     else x.val = val;  
-     x.size = 1 + size(x.left) + size(x.right);  
-     return x;  
- }
```

**Complessità di tempo: $O(h)$ nel caso peggiore
h altezza dell'ABR**



RICERCA DI UN ELEMENTO X IN UN ABR

- Se l'albero è vuoto, allora l'elemento x non è presente
- Altrimenti:
 - Si confronta x con la radice r dell'albero corrente;
se x è uguale ad r, allora l'elemento è stato trovato,
se x è minore di r, allora la ricerca di x
prosegue nel sottoalbero sinistro, altrimenti
la ricerca di x prosegue nel sottoalbero
destro.



**Complessità di tempo: $O(h)$ nel caso peggiore
h altezza dell'ABR**



RICERCA DELL'ELEMENTO MINIMO IN UN ABR

Come trovare l'elemento minimo in un ABR?

Qual è la complessità di tempo di tale operazione?



RICERCA DELL'ELEMENTO MINIMO IN UN ABR

```
public Key min() {  
    if (isEmpty()) throw new NoSuchElementException("calls min() with empty symbol table");  
    return min(root).key;  
}  
  
private Node min(Node x) {  
    if (x.left == null) return x;  
    else        return min(x.left);  
}
```



RICERCA DELL'ELEMENTO MASSIMO IN UN ABR

Come trovare l'elemento massimo in un ABR?

Qual è la complessità di tempo di tale operazione?



INSERIMENTO IN UN ABR

- Caso 1 - albero vuoto:
 - Il nuovo nodo diventa una foglia dell'albero
- Caso 2 - l'albero non è vuoto:
 - Si confronta il nodo x da inserire con la radice r dell'albero corrente. Se x è minore o uguale a r , allora x si inserirà (ricorsivamente nel sottoalbero sinistro), altrimenti nel sottoalbero destro.

**Complessità di tempo: $O(h)$ nel caso peggiore
 h altezza dell'ABR**



COSTO DELLE OPERAZIONI

- Il costo delle operazioni precedentemente descritte dipende dalla forma dell'albero e quindi dall'ordine con cui vengono inseriti gli elementi.
- Quanto costano queste operazioni nel caso peggiore, per un albero con N nodi?
- E nel caso migliore?
- E nel caso medio?



INSERIMENTO CASUALE

- Se le N chiavi sono inserite in un ordine casuale:

PROPOSIZIONE: Le operazioni di inserimento e ricerca richiedono, in media, circa $1.39 \log N$ confronti.

algoritmo (struttura dati)	caso peggiore (dopo N inserzioni)		caso medio (dopo N inserzioni casuali)		operazioni ordinate efficienti?
	search	insert	search hit	insert	
ricerca sequenziale (lista concatenata non ordinata)	N	N	$N/2$	N	no
ricerca binaria (array ordinato)	$\log N$	$2N$	$\log N$	N	si
albero binario di ricerca (BST)	N	N	$1.39 \log N$	$1.39 \log N$	si

Min, max, rank, select...

Tempi di calcolo (in caso di strutture senza ripetizioni)



ORDINAMENTO TRAMITE ABR

- Dato un insieme di elementi memorizzati in un ABR
 - è possibile ottenere l'elenco degli elementi ordinati in ordine crescente?
 - Sì, tramite la visita in ordine simmetrico
 - è possibile ottenere l'elenco degli elementi ordinati in ordine decrescente?
 - Sì, tramite la visita in ordine simmetrico inverso



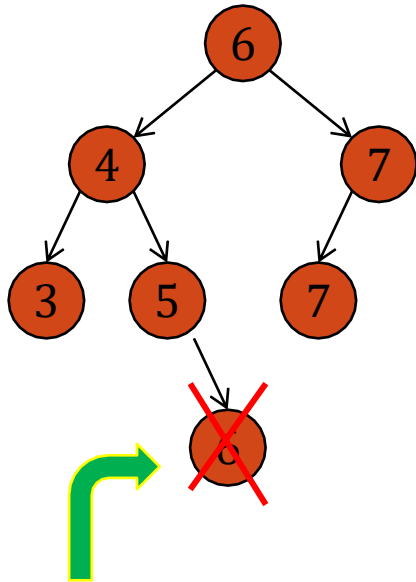
CANCELLAZIONE DI UN NODO IN UN ABR

- Si distinguono tre casi
 - Il nodo da cancellare è una foglia
 - Il nodo da cancellare ha un solo figlio
 - Il nodo da cancellare ha due figli
- Bisogna effettuare l'operazione mantenendo la struttura di ABR



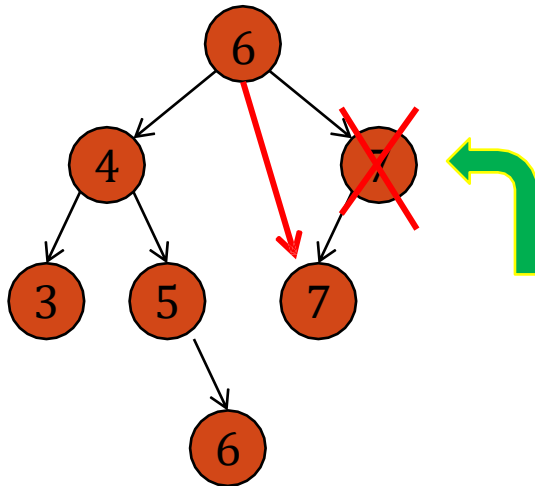
CANCELLAZIONE DI UN NODO IN UN ABR

- Primo caso: il nodo da cancellare è una foglia
 - Si elimina la foglia
 - L'albero rimane un ABR



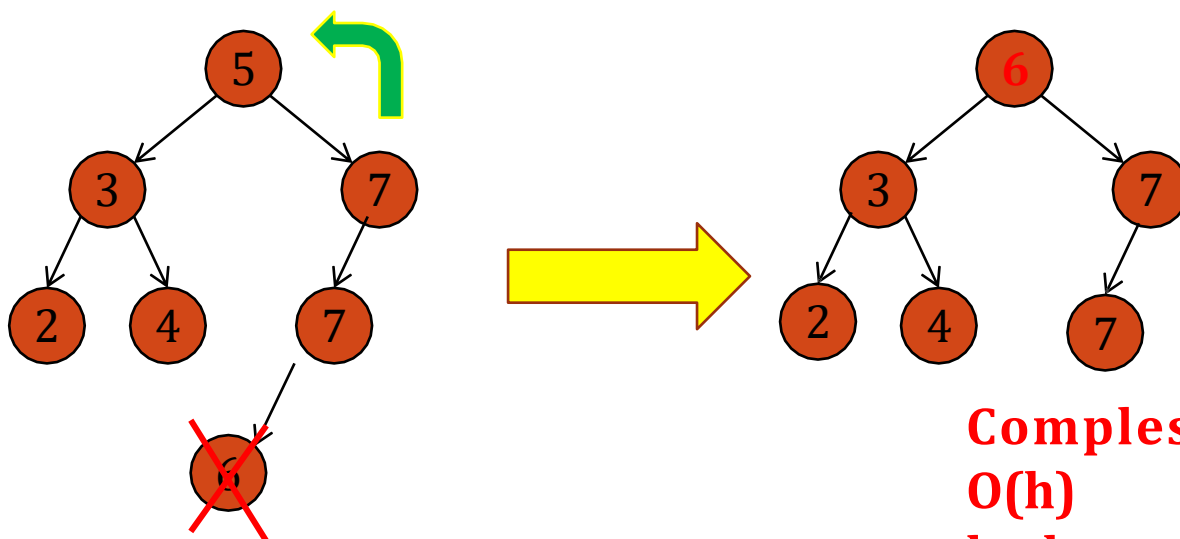
CANCELLAZIONE DI UN NODO IN UN ABR

- Secondo caso: il nodo da cancellare ha un figlio
 - Si fa puntare il padre del nodo al figlio del nodo da cancellare
 - L'albero rimane un ABR



CANCELLAZIONE DI UN NODO IN UN ABR

- Terzo caso: il nodo da cancellare ha due figli
 - Si cerca il minimo del sottoalbero destro e il suo valore si copia nel nodo da cancellare
 - Si cancella il minimo (sarà una foglia o un nodo con un figlio)
 - L'albero rimane un ABR



Complessità di tempo:
 $O(h)$
 h altezza dell'ABR



Cancellazione di un nodo in un ABR

```
- public void delete(Key key) {  
-     if (key == null) throw new  
IllegalArgumentException("calls delete() with a null  
key");  
-     root = delete(root, key);  
- }  
  
- private Node delete(Node x, Key key) {  
-     if (x == null) return null;  
-     int cmp = key.compareTo(x.key);  
-     if (cmp < 0) x.left = delete(x.left, key);  
-     else if (cmp > 0) x.right = delete(x.right, key);  
-     else {  
-         if (x.right == null) return x.left;  
-         if (x.left == null) return x.right;  
-         Node t = x;  
-         x = min(t.right);  
-         x.right = deleteMin(t.right);  
-         x.left = t.left;  
-     }  
-     x.size = size(x.left) + size(x.right) + 1;  
-     return x;  
- }
```



COSTO DELLE TRE OPERAZIONI: INSERIMENTO, RICERCA E CANCELLAZIONE

- Il costo delle operazioni di “dizionario” è $O(h)$ nel caso peggiore, dove h è l'altezza dell'albero.
- Nel caso in cui l'ABR è un albero lineare il costo è $O(n)$ – CASO ALBERO PESSIMO
- Nel caso in cui l'ABR è un albero completo il costo è $O(\log n)$ – CASO ALBERO OTTIMO
- Si può dimostrare che l'altezza media di un albero binario di ricerca costruito in modo casuale con n chiavi distinte è $O(\log(n))$ – CASO ALBERO MEDIO



Esercizi su ABR

- Si descriva un metodo che dato un elemento G , restituisca il numero di nodi con etichetta maggiore di G . Tale metodo deve restituire il valore utilizzando le proprietà dell'ABR, ovvero evitando di esaminare tutti i nodi dell'albero.
- Si aggiunga un metodo che presi in input due elementi M e N , con M minore o uguale a N , stampi tutti gli elementi compresi tra M e N



Esercizi su ABR

Si supponga di avere per ogni nodo l'attributo «parent»

Scrivere un metodo per la ricerca del predecessore di un elemento dell'albero binario di ricerca



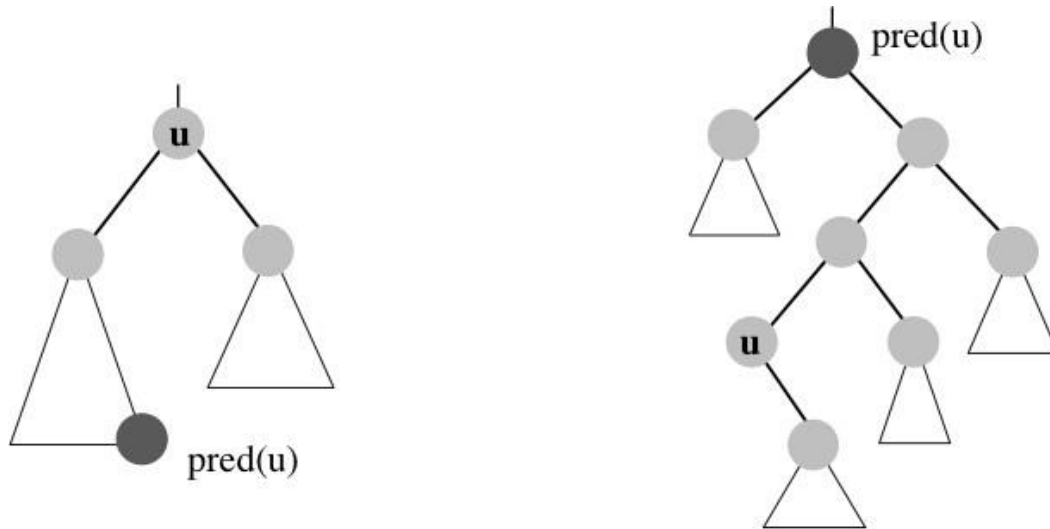
RICERCA DEL PREDECESSORE

Si supponga di avere per ogni nodo l'attributo «parent»

Descrivere un metodo per la ricerca del predecessore di un elemento dell'albero binario di ricerca



RICERCA DEL PREDECESSORE



- algoritmo** $\text{pred}(\text{nodo } u) \rightarrow \text{nodo}$
1. **if** (u ha figlio sinistro $\text{sin}(u)$) **then**
 2. **return** $\text{max}(\text{sin}(u))$
 3. **while** ($\text{parent}(u) \neq \text{null}$ e u è figlio sinistro di suo padre) **do**
 4. $u \leftarrow \text{parent}(u)$
 5. **return** $\text{parent}(u)$

